Revisit to CPE233 (Digital Design)

Authors:
WhisperingRock

Professor:

Class
Quarter
July 13, 2026

Contents

Note : All assignments are in the **CPE 233 Lab Manual**.

0.1 Hardware

0.2 Software

0.2.1 A1 - Intro to Assembly Lang

Note : Assume caller/callee does not need to preserve their respective registers in order to use them. KISS method applied for assignment 1.

Part 1

"Read three 16-bit unsigned values consecutively from SWITCHES (address 0x11000000), add them together, and output the result to LEDS (address 0x11000020). Assume the input values are 16-bit unsigned values. (Assume it is possible for the switches to change between each time they are read)"

Assuming the LEDS input value is limited to 16-bit unsigned as well.

Flowchart

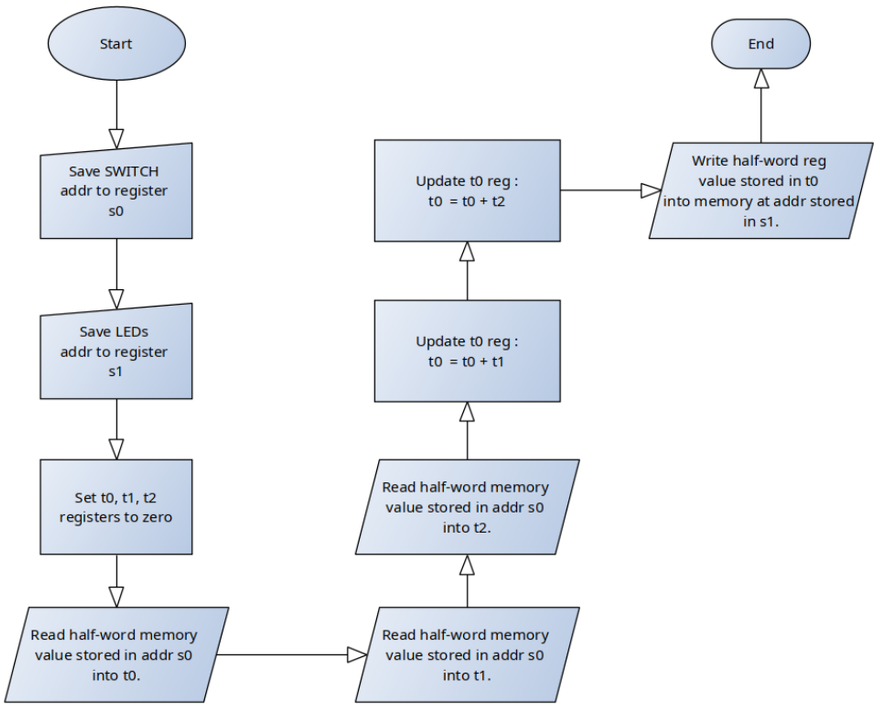

Figure 1: Flow Chart for Part 1

Verification Simulations

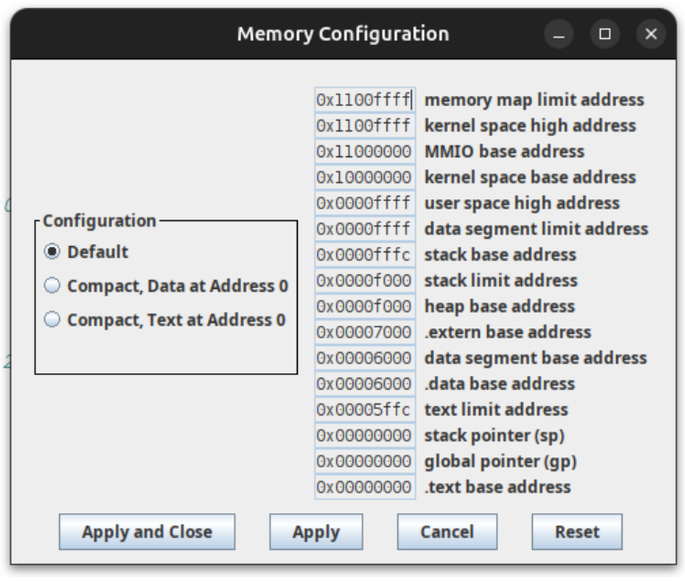

Figure 2: RARS Memory Configuration

```
20
21  #.equ SWITCHES_H,    0x11000 # upper 20 bits
22  #.equ SWITCHES_L,    0x000   # lower 12 bits
23  .equ SWITCHES_H,     0x00006 # upper 20 bits (data seg at 0x00006000)
24  .equ SWITCHES_L,     0x000   # lower 12 bits
25
26  #.equ LEDS_H,        0x11000 # upper 20 bits
27  #.equ LEDS_L,        0x020   # lower 12 bits
28  .equ LEDS_H,         0x00006 # upper 20 bits (data seg at 0x00006020)
29  .equ LEDS_L,         0x020   # lower 12 bits
30
```

Figure 3: Address Correction to work with RARS

- **Test Case 1**

```
33  # TC1 : Word Clip
34  .equ READ1_H,        0x00001 # switch first read (0x1234)
35  .equ READ1_L,        0x234
36
37
38  .equ READ2_H,        0x12345 # switch second read (0x5678)
39  .equ READ2_L,        0x678
40
41  .equ READ3_H,        0x11111 # switch third read (0x1111)
42  .equ READ3_L,        0x111
43  #    answer= 0x79BD
44
```

Figure 4: TC1 given inputs

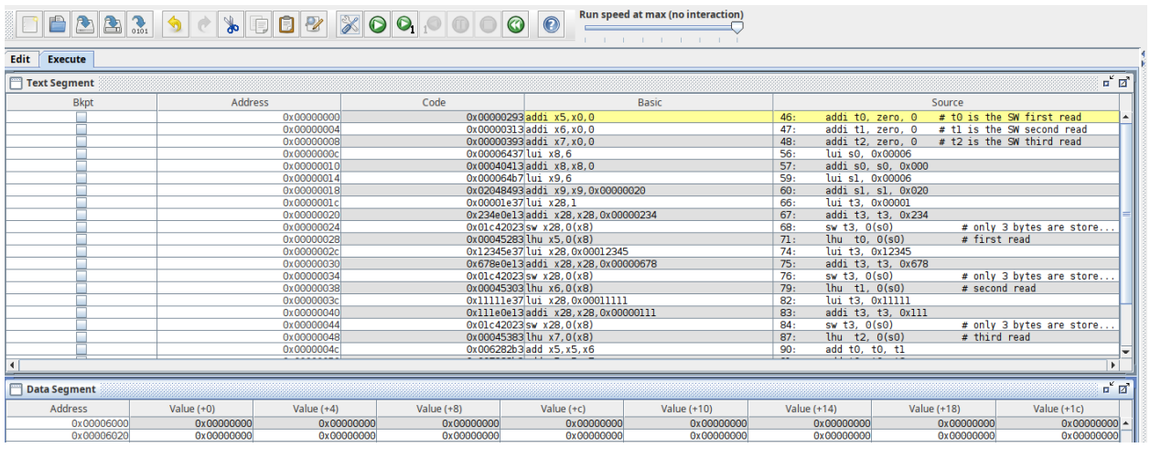

Figure 5: Data segment before executing instructions

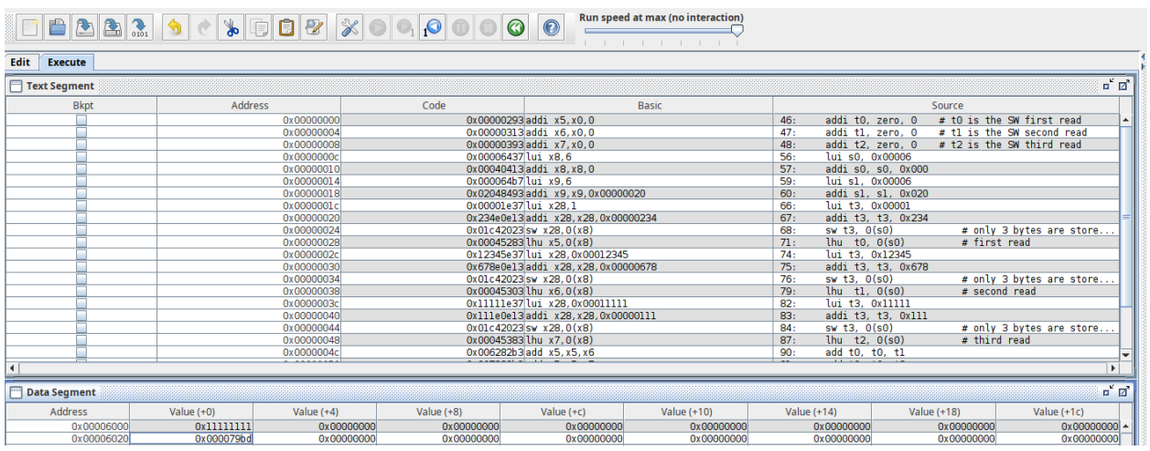

Figure 6: Data segment after executing instructions

Result at LEDS address in 6 matches the answer in 4.

- **Test Case 2**

```
46  # TC2 : Maxed
47  .equ READ1_H,        0x00005 # switch first read (0x5555)
48  .equ READ1_L,        0x555
49
50
51  .equ READ2_H,        0x00005 # switch second read (0x5555)
52  .equ READ2_L,        0x555   # sign extended
53
54  .equ READ3_H,        0x00005 # switch third read (0x5555)
55  .equ READ3_L,        0x555
56  #   answer= 0xFFFF
```

Figure 7: TC2 given input

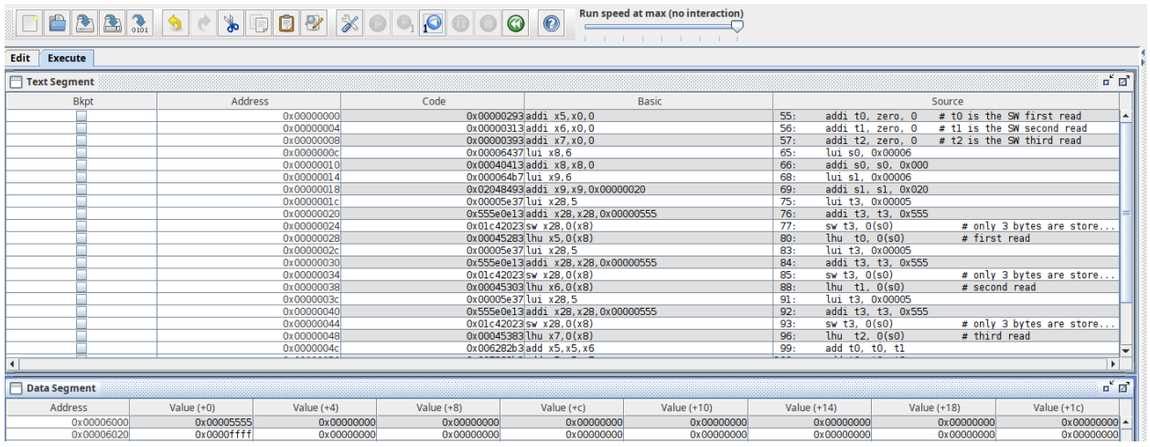

Figure 8: Data segment after executing instructions

Result at LEDS address in 8 matches the answer in 7.

- **Test Case 3**

```
59  # TC3 : Overflow
60  .equ READ1_H,        0x00005 # switch first read (0x5555)
61  .equ READ1_L,        0x555
62
63  .equ READ2_H,        0x00005 # switch second read (0x5555)
64  .equ READ2_L,        0x555   # sign extended
65
66
67  .equ READ3_H,        0x00005 # switch third read (0x5557)
68  .equ READ3_L,        0x557
69  #   answer= 0x0001
```

Figure 9: TC3 given input

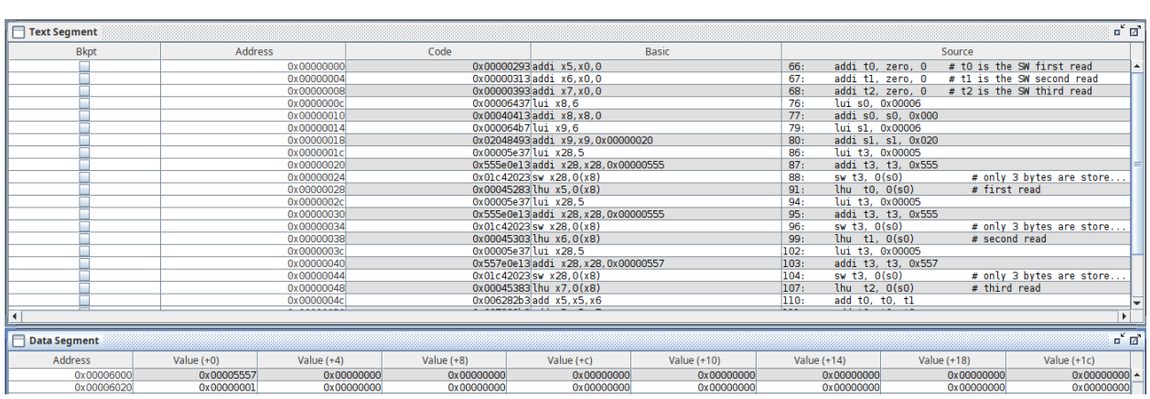

Figure 10: Data segment after executing instructions

Result at LEDS address in 10 matches the answer in 9.

- **Test Case 4**

```
71  # TC4 : zero
72  .equ READ1_H,        0x00000 # switch first read (0)
73  .equ READ1_L,        0x0
74
75
76  .equ READ2_H,        0x00000 # switch second read (0)
77  .equ READ2_L,        0x0 # sign extended
78
79  .equ READ3_H,        0x00000 # switch third read (0)
80  .equ READ3_L,        0x0
81  #    answer= 0x0000
82  "
```

Figure 11: TC4 given input

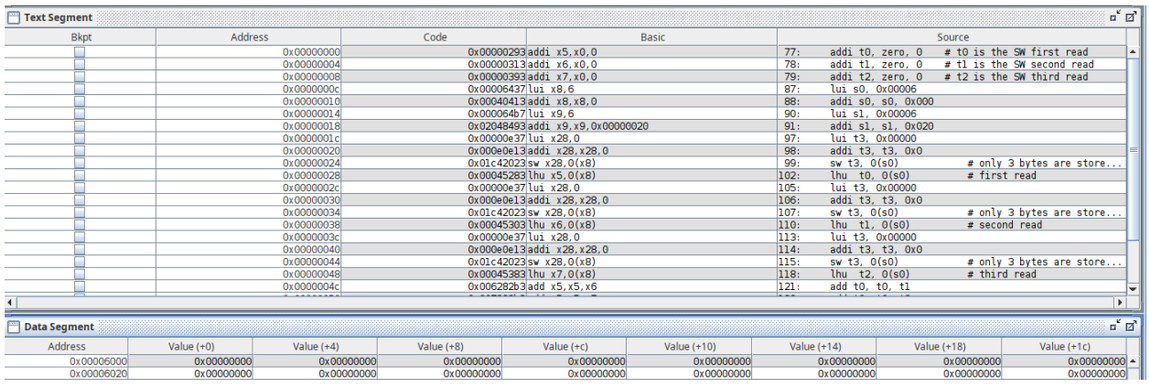

Figure 12: Data segment after executing instructions

Result at LEDS address in 12 matches the answer in 11.

- **Test Case 5**

```
84  # TC5 : MAXXXED
85  .equ READ1_H,        0x0000F # switch first read (0xFFFF)
86  .equ READ1_L,        -1
87
88  .equ READ2_H,        0xFFFFF # switch second read (0xFFFF)
89  .equ READ2_L,        -1
90
91
92  .equ READ3_H,        0x00000 # switch third read (0)
93  .equ READ3_L,        0
94  #    answer= 0xFFFE
```

Figure 13: TC5 given input

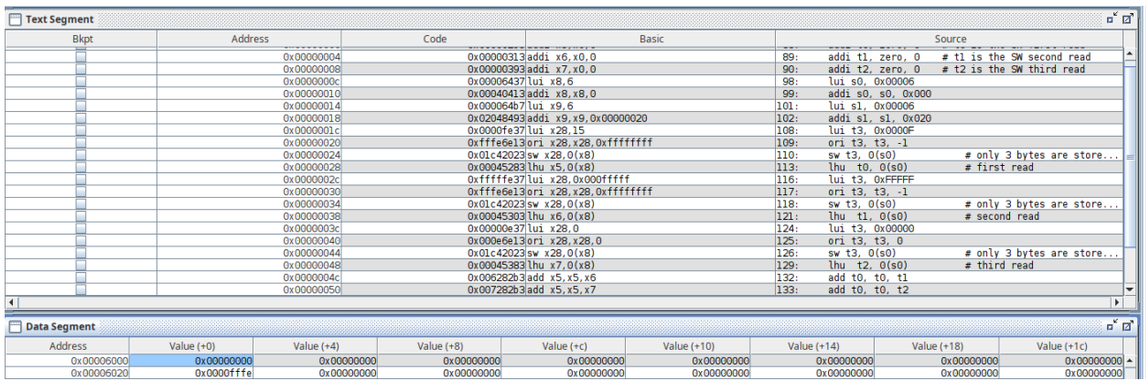

Figure 14: Data segment after executing instructions

Result at LEDS address in 14 matches the answer in 13.

- **Test Case 6**

```
96
97  # TC6 : All highs into full
98  .equ READ1_H,        0x00008 # switch first read
99  .equ READ1_L,        0x000
100
101 .equ READ2_H,        0x00008 # switch second read
102 .equ READ2_L,        0x000
103
104
105 .equ READ3_H,        0x00008 # switch third read
106 .equ READ3_L,        0x000
107 #    answer= 0x8000 (0x18000 as full word though)
```

Figure 15: TC6 given input

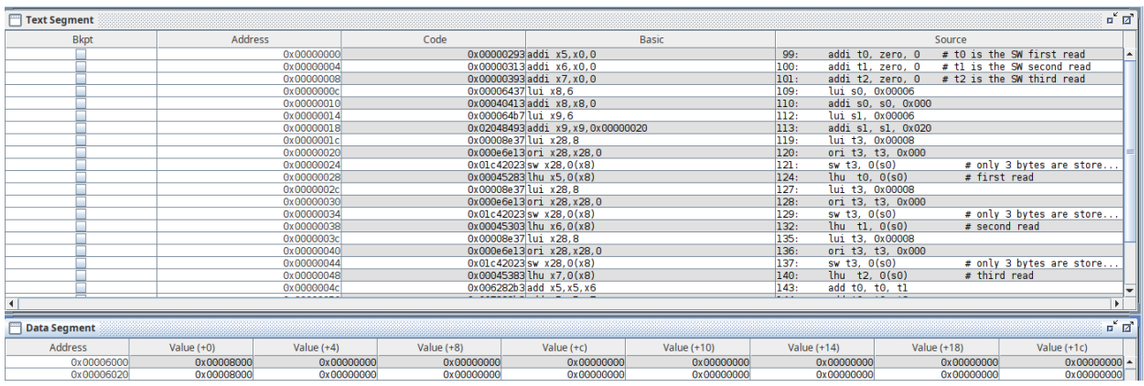

Figure 16: Data segment after executing instructions

Result at LEDS address in 16 matches the answer in 15.

Assembly Source Code

Listing 1: Question 1 of SW1

```
1  # Author : WhisperingRock
2  #
3  # Purpose : Read from the SWITCHES addr in memory three consecutive
4  #      times, summing the unsigned half-word reads and writing
5  #      to the LEDS addr.
6  #
7  # Notes :
8  #      - Assume caller/calle has no need to preserve reg values.
9  #        KISS method for this assignment
10 #
11 #      - Split 32-bit addresses according to instr imm handling.
12
13
14
15
16 .global main
17
18 #.equ SWITCHES_H, 0x11000 # upper 20 bits
19 #.equ SWITCHES_L, 0x000 # lower 12 bits
20 .equ SWITCHES_H,  0x00006 # upper 20 bits (data seg at 0x00006000)
21 .equ SWITCHES_L,  0x000 # lower 12 bits
22
23 #.equ LEDS_H,    0x11000 # upper 20 bits
24 #.equ LEDS_L,    0x020 # lower 12 bits
25 .equ LEDS_H,     0x00006 # upper 20 bits (data seg at 0x00006020)
26 .equ LEDS_L,     0x020 # lower 12 bits
27
28 # ~~~~~~~~ Test Cases ~~~~~~~~
29 # TC1 : Word Clip
30 #.equ READ1_H,     0x00001 # switch first read (0x1234)
31 #.equ READ1_L,     0x234
32
33 #.equ READ2_H,     0x12345 # switch second read (0x5678)
34 #.equ READ2_L,     0x678
35
36 #.equ READ3_H,     0x11111 # switch third read (0x1111)
37 #.equ READ3_L,     0x111
38 # answer= 0x79BD
39
40 # TC2 : Maxed
41 #.equ READ1_H,     0x00005 # switch first read (0x5555)
```

```
#.equ READ1_L,    0x555

#.equ READ2_H,    0x00005 # switch second read (0x5555)
#.equ READ2_L,    0x555 # sign extended

#.equ READ3_H,    0x00005 # switch third read (0x5555)
#.equ READ3_L,    0x555
# answer= 0xFFFF

# TC3 : Overflow
#.equ READ1_H,    0x00005 # switch first read (0x5555)
#.equ READ1_L,    0x555

#.equ READ2_H,    0x00005 # switch second read (0x5555)
#.equ READ2_L,    0x555 # sign extended

#.equ READ3_H,    0x00005 # switch third read (0x5557)
#.equ READ3_L,    0x557
# answer= 0x0001

# TC4 : zero
#.equ READ1_H,    0x00000 # switch first read (0)
#.equ READ1_L,    0x0

#.equ READ2_H,    0x00000 # switch second read (0)
#.equ READ2_L,    0x0 # sign extended

#.equ READ3_H,    0x00000 # switch third read (0)
#.equ READ3_L,    0x0
# answer= 0x0000

# TC5 : MAXXXED
#.equ READ1_H,    0x0000F # switch first read (0xFFFF)
#.equ READ1_L,    -1

#.equ READ2_H,    0xFFFFF # switch second read (0xFFFF)
#.equ READ2_L,    -1

#.equ READ3_H,    0x00000 # switch third read (0)
#.equ READ3_L,    0
# answer= 0xFFFE

# TC6 : All highs into full
.equ READ1_H,   0x00008 # switch first read
.equ READ1_L,     0x000

.equ READ2_H,     0x00008 # switch second read
.equ READ2_L,     0x000

.equ READ3_H,   0x00008 # switch third read
.equ READ3_L,   0x000
# answer= 0x8000 (0x18000 as full word though)
# ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

main:

  # ~~~~ register init ~~~~
  addi t0, zero, 0  # t0 is the SW first read
  addi t1, zero, 0  # t1 is the SW second read
  addi t2, zero, 0  # t2 is the SW third read
          # t3 is a dummy variable for testing

          # s0 contains the SWITCH addr
          # s1 contains the LEDS addr


  # ~~~~ load addr into registers ~~~~
  lui s0, SWITCHES_H
  addi s0, s0, SWITCHES_L

  lui s1, LEDS_H
  addi s1, s1, LEDS_L


  # ~~~~ read SWITCHES and sum ~~~~

  # ~~ behind the scenes : switch mem changes value ~~
  lui t3, READ1_H
  ori t3, t3, READ1_L
  sw t3, 0(s0)      # only 3 bytes are stored (0x234)
  # ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

  lhu  t0, 0(s0)      # first read

  # ~~ behind the scenes : switch mem changes value ~~
  lui t3, READ2_H
  ori t3, t3, READ2_L
  sw t3, 0(s0)      # only 3 bytes are stored (0x678)
  # ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

  lhu  t1, 0(s0)      # second read
```

```
133
134 # ~~~~ behind the scenes : switch mem changes value ~~~~
135 lui t3, READ3_H
136 ori t3, t3, READ3_L
137 sw t3, 0(s0)       # only 3 bytes are stored (0x111)
138 # ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
139
140 lhu  t2, 0(s0)       # third read
141
142 # ~~~~ sum values ~~~~
143 add t0, t0, t1
144 add t0, t0, t2
145
146
147 # ~~~~ Write the unsigned half word to LEDS mem addr
148 sh t0, 0(s1)
149
150
151 # ~~~~ return with exit status ~~~~
152 li a7, 10    # load system call number for exit()
153 li a0, 0     # Load the exit status code ( EXIT_SUCCESS )
154 ecall      # Make the system call
```

Part 2

"Read an input from SWITCHES (address 0x11000000). Assume the input is a 16-bit signed value in 2's complement (RC). Change the sign of the input and output the result to LEDS (address 0x11000020). The result should also be a 16-bit signed value in 2's complement (RC)."

Flowchart

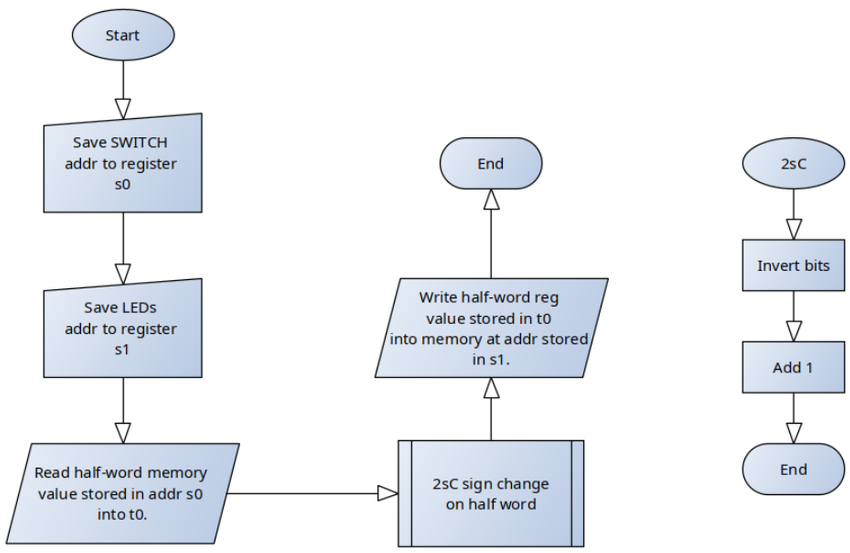

Figure 17: Flow Chart for Part 2

Verification Simulations
Assembly Code

0.3 Appendix